1 Problem Statement

Function minimization is an integral part of processes in fields such as Machine Learning and Quantum Mechanics. Most of the functions considered are too complex to minimize using high-order numerical methods, such as Newton's method. We explore Gradient Descent (GD), a first-order method that converges slowly. We then focus on Nesterov's Accelerated Gradient (NAG), a first-order method with much faster convergence than GD.

In this project, we consider this problem: For $f, g : \mathbb{R}^n \to \mathbb{R}$, find

$$\min_{x \in \mathbb{R}^n} f(x) \tag{1}$$

OR

$$\min_{x \in \mathbb{R}^n} f(x) + g(x). \tag{2}$$

In the first case, it is assumed that f is L-smooth– a property that is explained in detail in the following section. On the other hand, the second case still takes f to be L-smooth while g is NOT necessarily smooth [1, Section 2.2].

2 Assumptions/Hypotheses

For this project, we considered two conditions:

Convexity: According to [3], for any function $f : \mathbb{R}^n \to \mathbb{R}$, this is defined by

$$\forall x, y \in \mathbb{R}^n, \forall \lambda \in [0, 1] : f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda) f(y).$$

If f is differentiable, then convexity can also be defined as

$$\forall x, y \in \mathbb{R}^n : f(y) \geq f(x) + \nabla f(x)^T (y - x)$$

and if f is twice differentiable, convexity is

$$\nabla^2 f(x) \succeq 0, \forall x \in \mathbb{R}^n.$$

Figure 1 is a visual representation of a convex function $f : \mathbb{R} \to \mathbb{R}$:

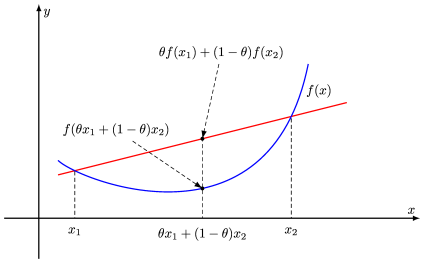

Figure 1: Convex Function as seen in [5].

Furthermore, f is μ-strongly convex if, for $\mu > 0$, for all $x, y \in \mathbb{R}^n$ and for all $\lambda \in [0, 1]$:

$$\mu \frac{\lambda(1-\lambda)}{2}\|x-y\|^2 + f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$$

if f is differentiable, f is μ-strongly convex if

$$f(y) \geq f(x) + \nabla f(x)^T(y-x) + \frac{\mu}{2}\|y-x\|^2, \forall x, y \in \mathbb{R}^n$$

if f is twice differentiable, strong convexity is defined by

$$\nabla^2 f(x) \succeq \mu I, \forall x \in \mathbb{R}^n.$$

L-smoothness: According to [3, 6], this is defined by

$$\exists L > 0 : \|\nabla f(x) - \nabla f(y)\| \leq L\|x-y\|, \forall x, y \in \mathbb{R}^n$$

if f is L-smooth, then

$$\exists L > 0 : f(y) \leq f(x) + \nabla f(x)^T(y-x) + \frac{L}{2}\|y-x\|^2, \forall x, y \in \mathbb{R}^n$$

if f is twice differentiable, f is L-smooth if

$$\nabla^2 f(x) \preccurlyeq LI, \forall x \in \mathbb{R}^n.$$

In all definitions, $\|\cdot\|$ and $\langle \cdot, \cdot \rangle$ denote the Euclidean norm and the Euclidean inner product, respectively, on $\mathbb{R}^n$. As mentioned, we study functions in which we always assume that f is L-smooth. We also explore when f is simply convex versus when it is μ-strongly convex. On the other hand, g is considered convex but not necessarily smooth.

3 Methods

3.1 Gradient Descent

For case (1), we used the plain Gradient Descent method. Given that the goal is to get as close to the minimum as possible, and as fast as possible, we start at any given point in the domain ($x_0 \in \mathbb{R}^n$) and move against the slope (∇f) at a certain step size (τ) each time; namely $x_{k+1} = x_k - \tau \nabla f(x_k)$. Figure 6 contains an example of gradient descent on a simple function, $f(x) = x^2$.

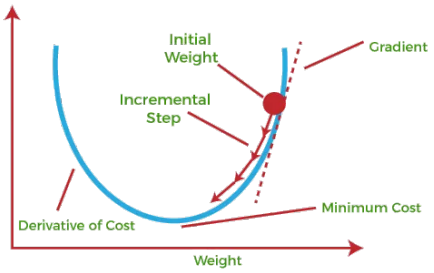

Figure 2: Gradient Descent on $f(x) = x^2$ as seen in [8].

This means that we have the following algorithm:

Algorithm 1: Gradient Descent

Input: $f : \mathbb{R}^n \to \mathbb{R}$, $x_0 \in \mathbb{R}^n$, $\tau > 0$, N_{max}, $\epsilon > 0$
Output: X: list of all x_k
$k = 0$;
while $k < N_{max}$ *and* $\|\nabla f(x_k)\|^2 > \epsilon$ **do**
  $x_{k+1} = x_k - \tau \nabla f(x_k)$;
  $k = k + 1$;
end

For this method, τ is constant and can be chosen arbitrarily with the only constraint being $0 < \tau \leq \frac{1}{L}$, where L is the constant for which f is L-smooth. To explain this we use the *Descent Lemma*, which stipulates that

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2}\|y - x\|^2; \forall x, y \in \mathbb{R}$$

If we consider $y = x_{k+1}$ and $x = x_k$ then, we have

$$f(x_{k+1}) \leq f(x_k) - \tau\|\nabla f(x_k)\|^2 + \frac{L\tau^2}{2}\|\nabla f(x_k)\|^2 = f(x_k) + \tau(\frac{L\tau}{2} - 1)\|\nabla f(x_k)\|^2$$

Since $\|\nabla f(x_k)\|^2 \geq 0$, we have $f(x_{k+1)} \leq f(x_k)$ if $\tau \leq \frac{2}{L}$. To optimize this, we would like $\tau(\frac{L\tau}{2} - 1)$ to be as negative as possible. The minimum is reached at $\tau = \frac{1}{L}$. But, since L is not always computable, $0 < \tau \leq \frac{1}{L}$ is sufficient. There is a process of generating τ iteratively named **backtracking** which we will see in Section 4.3.

With f μ-strongly convex, the error gap behaves such that [2]

$$f(x_k) - f^* \leq \left(1 + \frac{\mu}{L}\right)^k (f(x_0) - f^*)$$

If f is assumed to be only convex, the error gap is

$$f(x_k) - f^* \leq \frac{\|x_0 - x^*\|^2}{2\tau k}$$

where x^* is the minimizer of f, and $f^* = f(x^*)$. GD, here, converges at a rate no worse than $O(\frac{1}{k})$ [3].

3.2 Proximal Gradient Descent

For case (2), we cannot directly apply GD since that requires the entire function to be smooth and g is not necessarily smooth. Here, we introduce the proximal gradient to minimize $g : \mathbb{R}^n \to \mathbb{R}$. The proximal gradient for g at iteration k is :

$$x_{k+1} = \text{prox}_{\tau g}(x_k),$$

where, for any $h : \mathbb{R}^n \to \mathbb{R}$, the proximal map is

$$\text{prox}_h(x) = \text{argmin}_{y \in \mathbb{R}^n}[h(y) + \frac{1}{2}\|y - x\|^2]. \tag{3}$$

For $f + g$, we perform GD in f and the proximal gradient in g to yield $x_{k+1} = \text{prox}_{\tau g}(x_k - \tau \nabla f(x_k))$ Therefore, the algorithm used for proximal gradient is

Proximal GD converges similarly to Gradient Descent, whether with the function μ-strongly convex ($O((1 - \frac{\mu}{L})^k)$) or only convex ($O(\frac{1}{k})$).

Algorithm 2: Proximal Gradient

Input: $f, g : \mathbb{R}^n \to \mathbb{R}$, $x_0 \in \mathbb{R}^n$, $\tau > 0$, N_{max}, $\epsilon > 0$
Output: X: list of all x_k
$k = 0$;
while $k < N_{max}$ *and* $\|\nabla f(x_k)\|^2 > \epsilon$ **do**
  $x_{k+1} = prox_{\tau g}(x_k - \tau \nabla f(x_k))$;
  $k = k + 1$;
end

At this point, it is interesting to note the relation of GD and Proximal GD with the Forward (FE) and Backward (BE) Euler methods, applied to discretize the initial value problem (IVP)

$$\begin{cases} \dot{X} = -\nabla f(X) \\ X(0) = x_0 \end{cases}$$

$$FE : \frac{x_k - x_{k-1}}{\tau} = -\nabla f(x_{k-1})$$
$$BE : \frac{x_k - x_{k-1}}{\tau} = -\nabla f(x_k)$$

The similarity between Forward Euler and GD is easily noticeable. As for the Proximal GD, if applied to f which is L-smooth, the x_k obtained from Backward Euler solves $\nabla f(x) + \frac{1}{\tau}(x - x_{k-1}) = 0$. This is the gradient of equation (3). This means that the x_k obtained is the one that minimizes the function.

3.3 Nesterov's Accelerated Gradient

The methods described above have a slow convergence rate. In 1983, Yurii Nesterov, a Russian mathematician, introduced accelerated gradient schemes. The following is an algorithm for a simple Nesterov's Accelerated Gradient (NAG) scheme [6, pages 717-718].

Algorithm 3: Simple NAG

Input: $f : \mathbb{R}^n \to \mathbb{R}$,$x_0 \in \mathbb{R}^n$, $\tau > 0$, N_{max}, $\epsilon > 0$
Output: X: list of all x_k
$k = 0$;
$y_0 = x_0$;
$\theta_0 = 1$;
while $k < N_{max}$ *and* $\|\nabla f(y_k)\|^2 > \epsilon$ **do**
  $x_{k+1} = y_k - \tau \nabla f(y_k)$;
  $\theta_{k+1} = \frac{1}{2} + \sqrt{\frac{1}{4} + \theta_k^2}$;
  $\alpha_{k+1} = \frac{\theta_k - 1}{\theta_{k+1}}$;
  $y_{k+1} = x_{k+1} + \alpha_{k+1}(x_{k+1} - x_k)$;
  $k = k + 1$;
end

According to [1], for NAG, the error gap behaves in this way:

$$f(x_k) - f^* \leq \frac{2\alpha \|x_0 - x^*\|^2}{\tau(k+1)^2}$$

for any $0 < \tau \leq \frac{1}{L}$ This means that NAG behaves, at worst, like $O(\frac{1}{k^2})$– much faster than simple Gradient schemes.

Here is an alternative scheme of NAG from [6]:

Algorithm 4: NAG with q

Input: $f : \mathbb{R}^n \to \mathbb{R}$, $x_0 \in \mathbb{R}^n$, $\tau \in \mathbb{R}$, and $q \in [0, 1]$, N_{max}, $\epsilon > 0$
Output: X: list of all x_k
$k = 0$;
$y_0 = x_0$;
$\theta_0 = 1$;
while $k < N_{max}$ *and* $\|\nabla f(y_k)\|^2 > \epsilon$ **do**
  $x_{k+1} = y_k - \tau \nabla f(y_k)$;
  θ_{k+1} solves $\theta_{k+1}^2 = (1 - \theta_{k+1})\theta_k^2 + q\theta_{k+1}$;
  $\alpha_{k+1} = \frac{\theta_k(1-\theta_k)}{(\theta_k^2+\theta_{k+1})}$;
  $y_{k+1} = x_{k+1} + \alpha_{k+1}(x_{k+1} - x_k)$;
  $k = k + 1$;
end

Similar to Gradient Descent, Algorithm 4 converges for any $\tau \leq \frac{1}{L}$. The variable q can be moved between 0 and 1. At $q = 1$, Gradient Descent re-emerges. Moreover, for f μ-strongly convex, $q^* = \mu/L$ is the optimal choice for convergence and it gives an error behavior of

$$f(x_k) - f^* \leq L\left(1 - \sqrt{\frac{\mu}{L}}\right)^k \|x_0 - x^*\|^2.$$

This makes q^* the best possible setting for this scheme to converge as fast as possible. When $q > q*$ then this gives slow, monotone convergence. On the other hand, $q < q*$ provides regular oscillations during convergence.
Furthermore, if $q = q*$ then α_k tends towards

$$\alpha* = \frac{1 - \sqrt{\mu/L}}{1 + \sqrt{\mu/L}}.$$

So, when $\alpha_k < \alpha*$ there is monotone, slow convergence and when $\alpha_k > \alpha*$ there are regular oscillations during convergence.

For a μ-strongly convex function, it might be dificult to compute the actual values of μ and L. There is an alternative to choosing $q*$: restarting. An example of restart that we focus on is the Adaptive restart. There are two Adaptive restart schemes: "Function" and "Gradient" restart schemes [6]. We focus on the "Function" restart; whenever $f(x_{k+1}) > f(x_k)$, then we restart. This means we reset θ_k so that α_k goes back to 0. This helps avoid the oscillations that are characteristic in the convergence graphs of Algorithm 3.

Algorithm 5: NAG with Adaptive Restart, Function Scheme

Input: $f : \mathbb{R}^n \to \mathbb{R}$, $x_0 \in \mathbb{R}^n$, $\tau \in \mathbb{R}$, and $q \in [0, 1]$, N_{max}, $\epsilon > 0$
Output: X: list of all x_k
$k = 0$;
$y_0 = x_0$;
$\theta_0 = 1$;
while $k < N_{max}$ *and* $\|\nabla f(y_k)\|^2 > \epsilon$ **do**
  $x_{k+1} = y_k - \tau \nabla f(y_k)$;
  if $f(x_{k+1}) > f(x_k)$ **then**
    set $y_{k+1} = x_{k+1}$ and $\theta_k = 1$;
  else
    implement the **while** loop of Algorithm 4
  end
end

4 Implementation

In this section, we minimize various functions using the methods outlined in the previous section. All algorithms are implemented in Python with the help of the *Numpy* library for mathematical operations and *matplotlib* for graphing.

4.1 Basic Algorithm Implementation

We consider the following examples:

Example 1. *Minimize*

$$f(x) = \rho \log \left(\sum_{i=1}^{m} \exp\big((a_i^T x - b_i)/\rho\big) \right)$$

.

Here, $f : \mathbb{R}^n \to \mathbb{R}$ and is an example of case 1 In this specific example, ρ "controls the smoothness of the function" [6]. We test the gradient schemes for different values of ρ. Here, a_i^T is a row vector of length n containing the i^{th} row of A, and b is an m-length vector; A, b and x_0 are randomly generated with the standard normal distribution. We take $n = 20, m = 100$. The step-size taken is $\tau = 1/L$, where $L = \lambda_{\max}(A^T A)$. The test was run for $\epsilon = 10^{-16}$. Finally, x^* is taken to be the last element of the list of x_k generated by the algorithm.

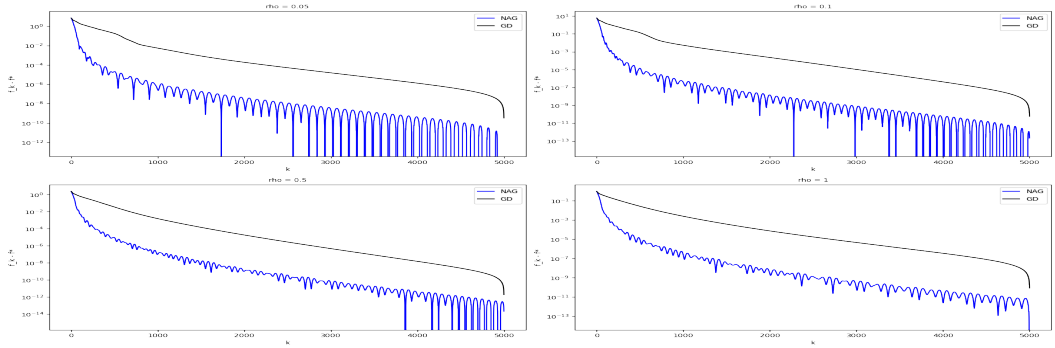

Figure 3: Minimization of the log-sum-exp function.

As can be observed, NAG reduces the error on the function faster than GD.

Example 2. *Minimize*

$$f(x) = x^T Ax + b^T x + 1, x \in \mathbb{R}^n.$$

In this example, also implementing case 1, A is an $n \times n$-matrix and b an n-length vector. A needs to be symmetric positive definite (SPD) for the function to have a unique global minimum. A must also be well-conditioned to avoid float-point errors that slow down convergence. In this example, we take $\tau = 1/L$, with $L = \lambda_{\max}(A)$. This function has a computable minimum, $x^* = -\frac{1}{2}A^{-1}b$. This function is also μ-strongly convex with $\mu = \lambda_{\min}(A)$. As such, we graph the error in the objective function for the iterates of Algorithms 1 and 3, along with Algorithm 4 with $q^* = \mu/L$.

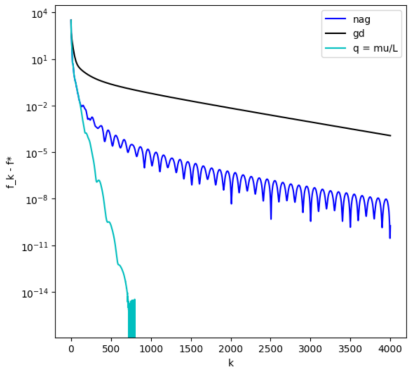

Figure 4: Minimization of μ-strongly convex function.

NAG behaves, as expected, better than Gradient Descent but using q^* gives the best convergence rate of all three.

Example 3. *Minimize*

$$\frac{1}{2}\|Ax - b\|_2^2 + \rho\|x\|_1.$$

This example from [6] implements case 2. Here, $\frac{1}{2}\|Ax - b\|_2^2$ is L-smooth with $L = \lambda_{\min}(A^T A)$ while $\rho\|x\|_1$ is not smooth. As we have seen, Proximal Gradient entails the minimization of $g(x) + \frac{1}{2\tau}\|x - x_k\|^2$. We use an explicit evaluation of the proximal map through the soft-threshold operator $\mathcal{T}_\alpha : \mathbb{R}^n \to \mathbb{R}^n$ defined by [1]

$$\mathcal{T}_\alpha(x)_i = \text{sign}(x_i)\max(|x_i| - \alpha, 0).$$

And so x_{k+1} is obtained as follows

$$x_{k+1} = \mathcal{T}_{\rho\tau}(x_k - \tau A^T(Ax_k - b)), \text{for GD}$$
$$x_{k+1} = \mathcal{T}_{\rho\tau}(y_k - \tau A^T(Ay_k - b)), \text{for NAG}$$

A is a randomly generated with standard normal distribution $m \times n$ matrix. $b = Ay + w$, where y is an n-length sparse vector with s non-zero entries while w is an m-length vector whose entries are sampled from a normal distribution $N(0, 0.1)$. Once again, $\tau = 1/L$. For this example, we took $m = 500, n = 2000, s = 100$ and $\rho = 1$.

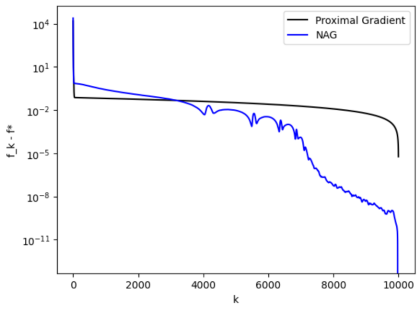

Figure 5: Minimization using Proximal Gradient.

This graph shows that it takes many more iterations to see a great difference in performance between NAG and Proximal Gradient than between NAG and GD. But as the iterations increase, NAG clearly decreases in error faster than Proximal Gradient.

4.2 Optimal Stepsize for Strongly-Convex Quadratic Minimization

There exist specific cases for which a stepsize that decreases the most the function at each step can be chosen. According to [4], for $f : \mathbb{R}^n \to \mathbb{R}$, $f(x) = \frac{1}{2}x^T Qx + b^T x$, where $Q \in \mathbb{R}^{n\times n}$ is SPD and $b \in \mathbb{R}^n$, the unique minimizer $\min_{\tau\in\mathbb{R}_{\geq 0}} f(x + \tau d)$, given x and d, is

$$\bar{\tau} = -\frac{\nabla f(x)^T d}{d^T Q d}.$$

This makes Gradient Descent converge much faster for these quadratic functions.

Example 4. *Minimize*

$$f(x) = \frac{1}{2}x^T Qx + c^T x + \gamma.$$

We used the values

$$Q = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & \delta \end{pmatrix}, c = (1,1,1,1)^T, \gamma = 0, x = (1,2,3,4)^T$$

We test this for $\delta = [10^{-1}, 10^{-2}, ..., 10^{-6}]$, as provided in [4].

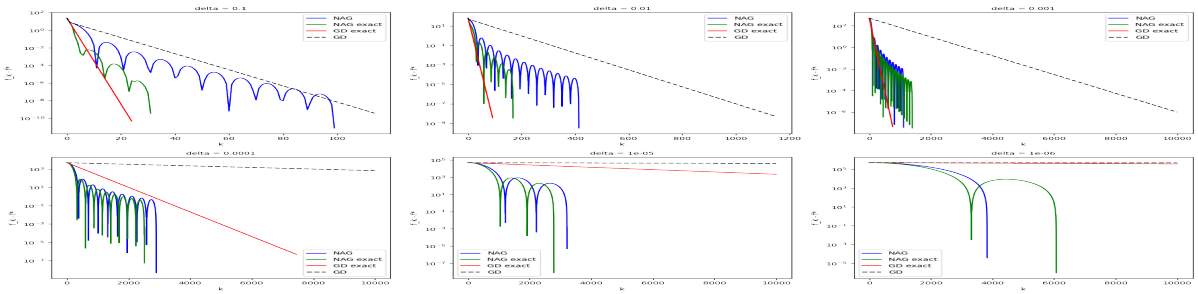

Figure 6: Minimization of a μ-strongly convex quadratic with optimal stepsize.

It can be observed that the exact stepsize makes GD and NAG behave better than with $\tau = 1/L$ for big values of δ. As δ decreases, however, the advantage of the exact stepsize is lost as both methods start behaving similarly to when $\tau = 1/L$, for $\delta = 10^{-6}$.

4.3 Backtracking

For a lot of the examples above, we use the Lipschitz constant L to obtain the step-size. However, this constant is not always known or easily computed. Backtracking is a way that allows as to calculate a constant L_k at each iteration, such that the objective value decreases. Essentially, we are given $L_0 > 0, \eta > 1$ and, at each step, we find the smallest integer i_k such that, with $L_k = \eta^{i_k} L_{k-1}$,

$$f(x_{k+1}) \leq f(x_k) + \langle (x_{k+1} - x_k), \nabla f(x_k) \rangle + \frac{L_k}{2} \|x_{k+1} - x_k\|^2, \text{for GD}$$
$$f(x_{k+1}) \leq f(y_k) + \langle (x_{k+1} - y_k), \nabla f(y_k) \rangle + \frac{L_k}{2} \|x_{k+1} - y_k\|^2, \text{for NAG}$$

In other words, given $\tau_0 > 0, 0 < \theta < 1$ we need to find $\tau_k = \theta^{i_k} \tau_{k-1}$ such that the above conditions are met (we have $L_k = \frac{1}{\tau_k}$).

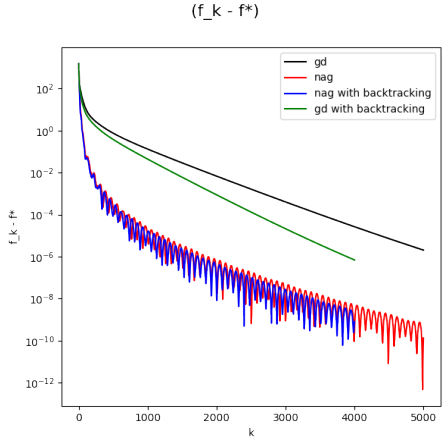

Figure 7: GD and NAG using backtracking on Example 2.

We can see that using backtracking for GD yields slightly improved convergence while it stays rather similar for NAG.

4.4 Restart

We implement adaptive function restart on Examples 2 and 3 using Algorithm 5 with $N_{max} = 5000, \epsilon = 10^{-16}$.

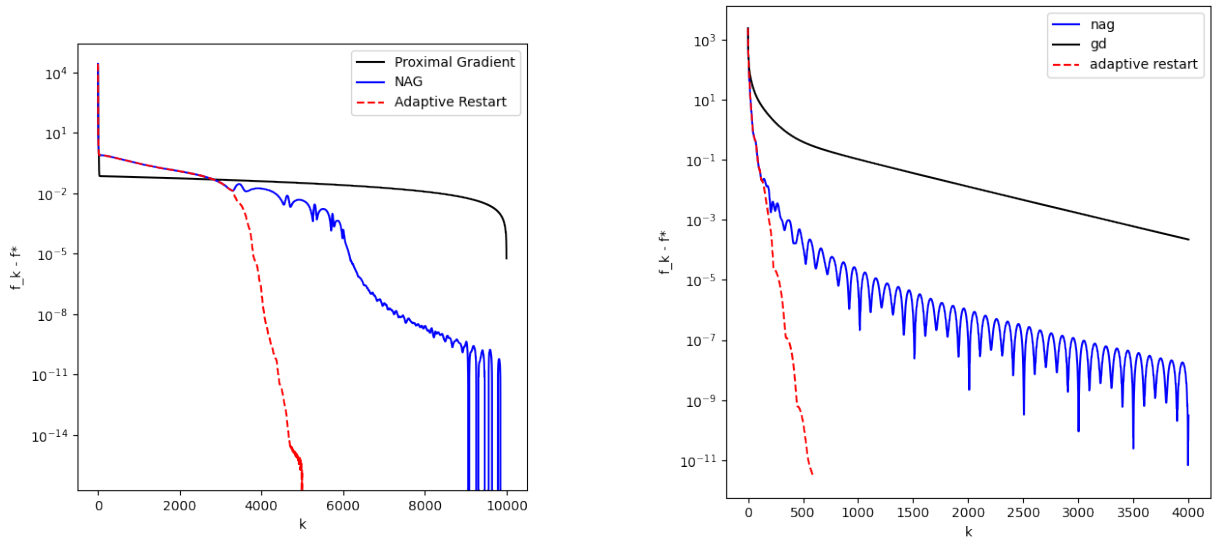

Figure 8: NAG with Adaptive Restart on Examples 2 and 3 respectively.

As can be seen, adaptive restart provides a convergence rate that is significantly faster than "simple" NAG. In addition to the above restart algorithm which is found in [6], we propose a slightly different scheme in which, when $f(x_{k+1} > f(x_k)$, we only set $y_{k+1} = x_k$. This scheme proved to have a convergence rate very similar to the one above.

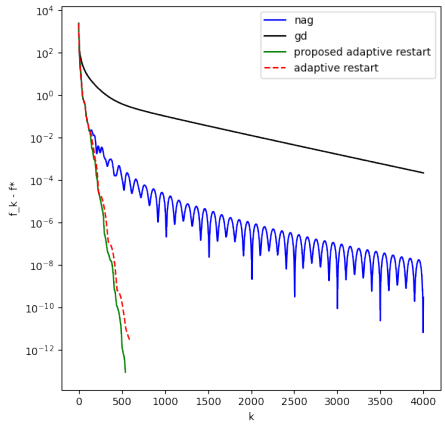

Figure 9: NAG with Adaptive Restart on Example 2.

We compared the behavior of the α from Algorithm 5 with this proposed scheme. Since θ is not being restarted in the proposed scheme, the α grows and approaches 1. Then, its growth slows down significantly but does not stop until 1, while [5] has an alpha that oscillates with each restart.

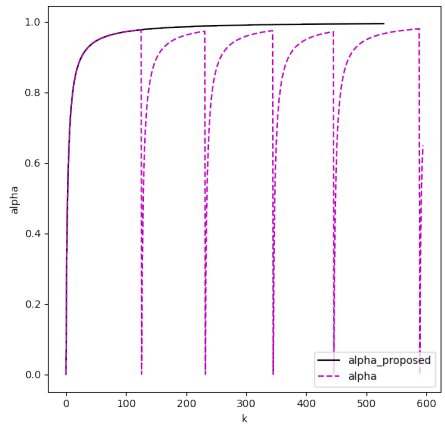

Figure 10: Comparison of the α for the adaptive restart schemes.

In this graph, the α for the proposed approach continues to grow very slowly towards 1. We also experimented with an approach that freezes the proposed α once the restart criterion is reached but this yielded the same convergence rate as with the proposed scheme where α keeps growing.

5 Functionals

So far, we have studied the behaviour of Gradient Descent and Nesterov's Accelerated Gradient on **convex** functions. In this section, we briefly turn our attention to non-convex functions to see in what circumstances GD and NAG converge, as well as in the convex cases. We apply the methods to functionals:

Example 5. *Minimize*

$$E(u) = \frac{1}{2}\int_\Omega (|u(x)|^2 - 1)^2 dx$$

Example 6. *Minimize*

$$E(u) = \frac{1}{2}\int_\Omega (|u'(x)|^2 - 1)^2 dx$$

Here, $E : X \to \mathbb{R}$ and $u : \mathbb{R} \to \mathbb{R}^\ltimes$, where X is some space of functions and Ω is a domain, which we consider to be $]0, 1[$ for these specific examples.
Let's define some operations: $|\cdot|$ denotes the Euclidean norm on $\mathbb{R}^n$ and $\langle x, y\rangle = x^T y$, for $x, y \in \mathbb{R}^n$ is the Euclidean inner product. We denote the inner product on X as $(\cdot,\cdot)_X$ and $DE(u)(v)$is the derivative of u in the direction of v, also known as the directional derivative defined as:

$$DE(u)(v) = \lim_{\epsilon \to 0} \frac{E(u + \epsilon v) - E(v)}{\epsilon}$$

and we have

$$\begin{aligned} DE(u)(v) &= \lim_{\epsilon \to 0} \frac{E(u + \epsilon v) - E(v)}{\epsilon} \\ E(u + \epsilon v) &= \frac{1}{2}\int_0^1 (|u + \epsilon v|^2 - 1)^2 dx \\ &= \frac{1}{2}\int_0^1 (|u|^2 + 2\epsilon < u, v > +\epsilon^2 |v|^2 - 1)^2 dx \end{aligned}$$

$$= \frac{1}{2}\int_0^1 (|u|^2-1)^2 dx + \int_0^1 (|u|^2-1)(2\epsilon < u, v > +\epsilon^2|v|^2)dx$$
$$+ \frac{1}{2}\int_0^1 (2\epsilon < u, v > +\epsilon^2|v|^2)^2 dx$$
$$= E(u) + \epsilon \int_0^1 2(|u|^2-1) < u, v > dx + \text{higher order terms in } \epsilon$$
$$DE(u)(v) = \lim_{\epsilon \to 0} \frac{\epsilon \int_0^1 2(|u|^2-1) < u, v > dx + \text{h.o.t} + E(u) - E(u)}{\epsilon}$$

$$DE(u)(v) = \int_0^1 2(|u|^2-1) < u, v > dx \tag{4}$$

With these operations, we define schemes for GD and NAG on the functional:

Algorithm 6: GD for functional

Input: $E : X \to \mathbb{R}$, $u_0 : \Omega \to \mathbb{R}^n$, $\tau > 0$, $tol > 0$
Output: U: list of all u_k
$k = 0$;
while $k < N_{max}$ *and* $E_\delta > tol$ **do**
  Find $\delta_{u_k} \in X$ such that $(\delta_{u_k})(v) = -\tau DE(u_k)(v), \forall v \in X$;
  $u_{k+1} = u_k + \delta_{u_k}$;
  $k = k + 1$;
end

Algorithm 7: NAG for functional

Input: $E : X \to \mathbb{R}$, $u_0 : \Omega \to \mathbb{R}^n$, $\tau > 0$, $tol > 0$
Output: U: list of all u_k
$k = 0$;
$v_0 = u_0$;
$\theta_0 = 1$;
while $k < N_{max}$ *and* $E_\delta > tol$ **do**
  Find $\delta_{v_k} \in X$ such that $(\delta_{v_k})(v) = -\tau DE(v_k)(v), \forall v \in X$;
  $u_{k+1} = v_k + \delta_{v_k}$;
  $\theta_{k+1} = \frac{1}{2} + \sqrt{\frac{1}{4} + \theta_k^2}$;
  $\alpha_{k+1} = \frac{\theta_k - 1}{\theta_{k+1}}$;
  $v_{k+1} = u_{k+1} + \alpha_{k+1}(u_{k+1} - u_k)$;
  $k = k + 1$;
end

To be able to find δ_{u_k}, we used the Finite Element Method(FEM) [7, Section 4.3]. The application of FEM relies on the "weak" form of the problem to be solved in Algorithms 6 and 7. In this weak form, we need to find $\delta_{u_k} \in X_0$ such that

$$\epsilon \int_0^1 (\delta_{u_k})' v' + \int_0^1 (\delta_{u_k}) v = -\tau DE(u_k)(v), \forall v \in X_0 \tag{5}$$

where $X_0 = \{v \in X | v(0) = v(1) = 0\}$.
We consider these spaces:

$$L^2(\Omega) = \{v : \Omega \to \mathbb{R} | \int_\Omega |v(x)|^2 dx < \infty\}$$
$$H^1(\Omega) = \{v : \Omega \to \mathbb{R} | v, v' \in L^2(\Omega)\}$$

After running the algorithms, these are the graphs comparing the convergence rates of GD and NAG:

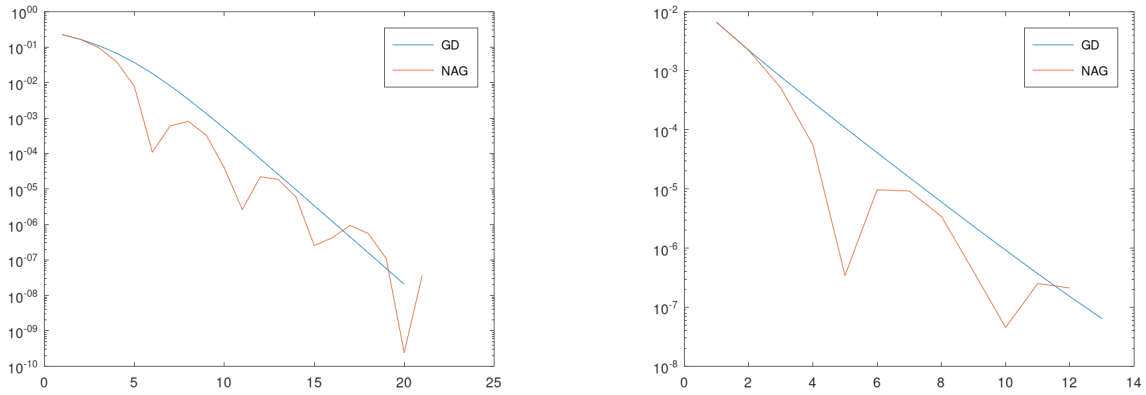

Figure 11: GD and NAG on Examples 5 and 6 respectively.

We use $u_0 = [r\cos(2\pi x); r\sin(2\pi x)]^T$ with $r = 1.1$. In both cases, NAG performs slightly better than GD, but the gap is not as substantial as in previously shown, convex cases. Furthermore, during testing, we observe that if $\epsilon = 0$, we take $X_0 = L^2$, which makes GD and NAG for case 5 converge and take a few iterations(13 and 10, respectively) to reach tolerance, while, for case 6, NAG diverges. On the other hand, if $\epsilon = 1$, we take $X_0 = H^1$, then GD and NAG converge in 13 and 12 iterations, respectively, for case 6 while they run for many more iterations for case 5.

References

[1] Amir Beck and Marc Teboulle, *A fast iterative shrinkage-thresholding algorithm for linear inverse problems*, SIAM Journal on Imaging Sciences **2** (2009), no. 1, 183–202.

[2] Stephen Boyd and Lieven Vandenberghe, *Convex optimization*, Cambridge University Press, 2004.

[3] Guillaume Garrigos and Robert M. Gower, *Handbook of convergence theorems for (stochastic) gradient methods*, 2024.

[4] Tim Hoheisel, *Optimization homework set no. 4*, 2020.

[5] Shailesh Kumar, *Topics in signal processing*, `https://tisp.indigits.com/convex_sets/convex_functions`, 2022.

[6] Brendan O'Donoghue and Emmanuel Candès, *Adaptive restart for accelerated gradient schemes*, Journal of the Society for the Foundations of Computational Mathematics (2013), 715–732.

[7] Alfio Quarteroni, *Numerical models for differential problems*, Modelling, Simulation and Applications, Springer Milano, 2014.

[8] Lakshya Ruhela, *Types of gradient descent*, Medium (2023).